Mask R-CNN 代码详解笔记 (Task 5)

> **前言**：作为一个大一新生，初次接触深度学习，我不仅跑通了代码，更尝试去理解每一行代码背后的逻辑。这份笔记是我对 Task 5 推理脚本 (`task5_infer.py`) 的逐行拆解与思考。

1. 环境准备与防报错

```
import os
# --- 修复 OMP: Error #15 ---
os.environ["KMP_DUPLICATE_LIB_OK"] = "TRUE"
```

- **作用**：这是专门针对 Windows 用户的“救命药”。在使用 PyTorch 和 Matplotlib 时，经常会因为 OpenMP 库冲突导致程序直接崩溃。加上这行代码，强制允许库副本存在，程序才能跑通。

2. 模型构建 (核心逻辑)

这部分代码体现了 **迁移学习 (Transfer Learning)** 的思想。

```
def get_model_instance_segmentation(num_classes):
    # 1. 加载骨架：使用 ResNet50 作为主干网络
    model = maskrcnn_resnet50_fpn(weights=None)

    # 2. 修改“检测头” (Box Predictor)
    in_features = model.roi_heads.box_predictor.cls_score.in_features
    # 替换预训练的 91 类分类器，改为我们的 num_classes (2类：背景+人)
    model.roi_heads.box_predictor = FastRCNNPredictor(in_features, num_classes)

    # 3. 修改“分割头” (Mask Predictor)
    in_features_mask = model.roi_heads.mask_predictor.conv5_mask.in_channels
    hidden_layer = 256
    # 替换预训练的分割头，使其输出对应类别的 Mask
    model.roi_heads.mask_predictor = MaskRCNNPredictor(in_features_mask,
hidden_layer, num_classes)

    return model
```

- **思考**：
  - Mask R-CNN 的原始模型是在 COCO 数据集（91种物体）上训练的。
  - 我们需要把它“微调”成只认识 **2 类**（0=背景，1=行人）。
  - **做法**：保留 ResNet50 提取特征的强大能力（不动它的身体），只把最后负责输出结果的“头”（分类器和分割器）换掉。

3. 加载训练好的“大脑”

```
# 判断是用 CPU 还是 GPU
device = torch.device('cuda') if torch.cuda.is_available() else
torch.device('cpu')

# 初始化一个空模型结构
model = get_model_instance_segmentation(num_classes)

# 加载权重文件
if os.path.exists("mask_rcnn_model.pth"):
    model.load_state_dict(torch.load("mask_rcnn_model.pth", map_location=device))
    model.to(device)
    model.eval() # <--- 关键！进入评估模式
```

- `model.load_state_dict(...)`：这是把训练好的参数（权重）填入到当前的空模型里。
- `model.eval()`：**非常重要！** 告诉 PyTorch “我现在是在考试，不是在训练”。这会冻结 Dropout 和 BatchNorm 层，保证预测结果稳定。

4. 进行预测 (Inference)

```
# 拿出一张图片
dataset = task5_dataset.PennFudanDataset(...)
img, _ = dataset[0]

# 真正开始预测
with torch.no_grad(): # 考试不需要算梯度（省内存）
    prediction = model([img.to(device)])
```

- **输入**：一张 Tensor 格式的图片。
- **输出 (`prediction`)**：这是一个字典，包含：
  - `boxes`: 目标的坐标 `[x1, y1, x2, y2]`。
  - `labels`: 目标的类别（全是 1，代表行人）。
  - `scores`: 置信度分数（0~1）。
  - `masks`: 每个目标的形状（0/1 矩阵）。

5. 结果筛选

```
# 筛选结果：只保留分数 > 0.5 的靠谱结果
keep = pred_scores > 0.5
pred_masks = pred_masks[keep]
pred_boxes = pred_boxes[keep]
```

- **逻辑**：模型可能会输出几千个框（很多是背景噪声）。我们只保留它认为“有 50% 以上把握是人”的结果。

6. 可视化绘制 (难点)

这一部分是为了让结果**“看得见”**，特别是 Mask 的半透明叠加。

6.1 准备底图

```
img_np = img.mul(255).permute(1, 2, 0).byte().numpy()
plt.imshow(img_np) # 画出原图
```

- PyTorch 的图是 `(C, H, W)` 且是 0-1 的浮点数，Matplotlib 需要 `(H, W, C)` 且是 0-255 的整数。这里做了转换。

6.2 画 Mask (红色半透明层)

```
# 创建一个全透明的图层
overlay = np.zeros((H, W, 4))
overlay[:, :, 0] = 1.0  # R通道设为1（红色）
# Alpha通道（透明度）：有人的地方设为 0.5（半透明），没人的地方设为 0（全透明）
overlay[:, :, 3] = np.where(combined_mask > 0.5, 0.5, 0.0)

plt.imshow(overlay) # 盖在原图上面
```

- **技术难点**：如果直接画 Mask，会把原图盖住。这里用 **RGBA** 技巧，只把 Mask 区域设为半透明红色，既能看到分割形状，又能看清背后的人。

6.3 画 Box (黄色框)

```
rect = patches.Rectangle((box[0], box[1]), box[2]-box[0], box[3]-box[1], ...)
ax.add_patch(rect)
```

- 根据预测的坐标画出黄色的矩形框。

7. 总结

这份代码实现了一个完整的 **检测 + 分割** 流程：

1. **Input**: 图片 -> Tensor。
2. **Model**: ResNet50 提取特征 -> RPN 生成候选框 -> RoIAlign 提取区域特征 -> Head 输出 Box/Mask。
3. **Output**: 解析预测结果 -> 过滤低分框 -> 绘制半透明 Mask -> 生成报告。